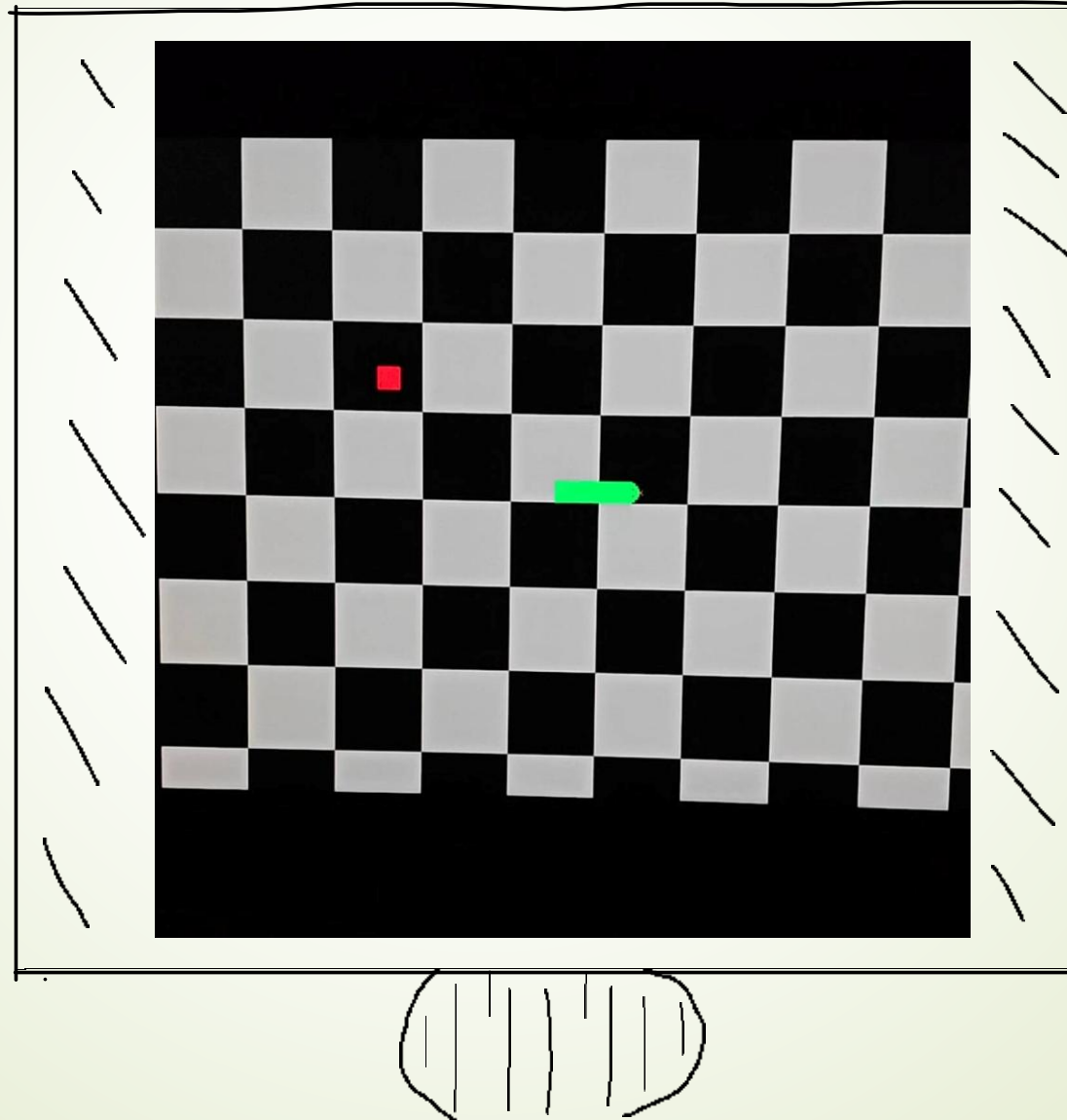
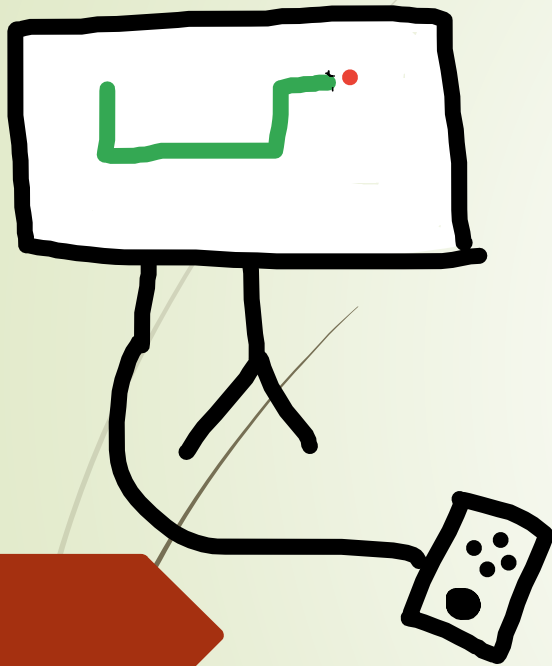
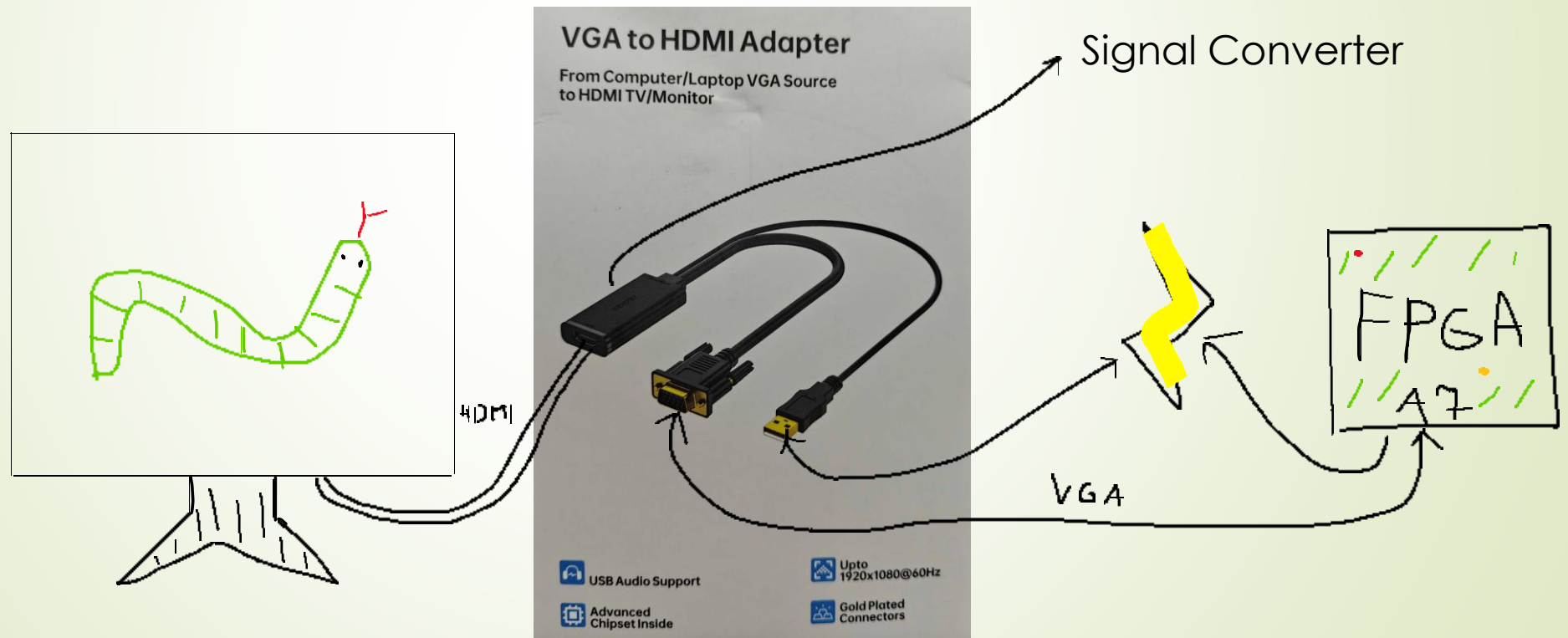


Video Subsystem in FproV2 SoC



Hardware Setup

- VGA-to-HDMI active converter which does exactly what one would think it does

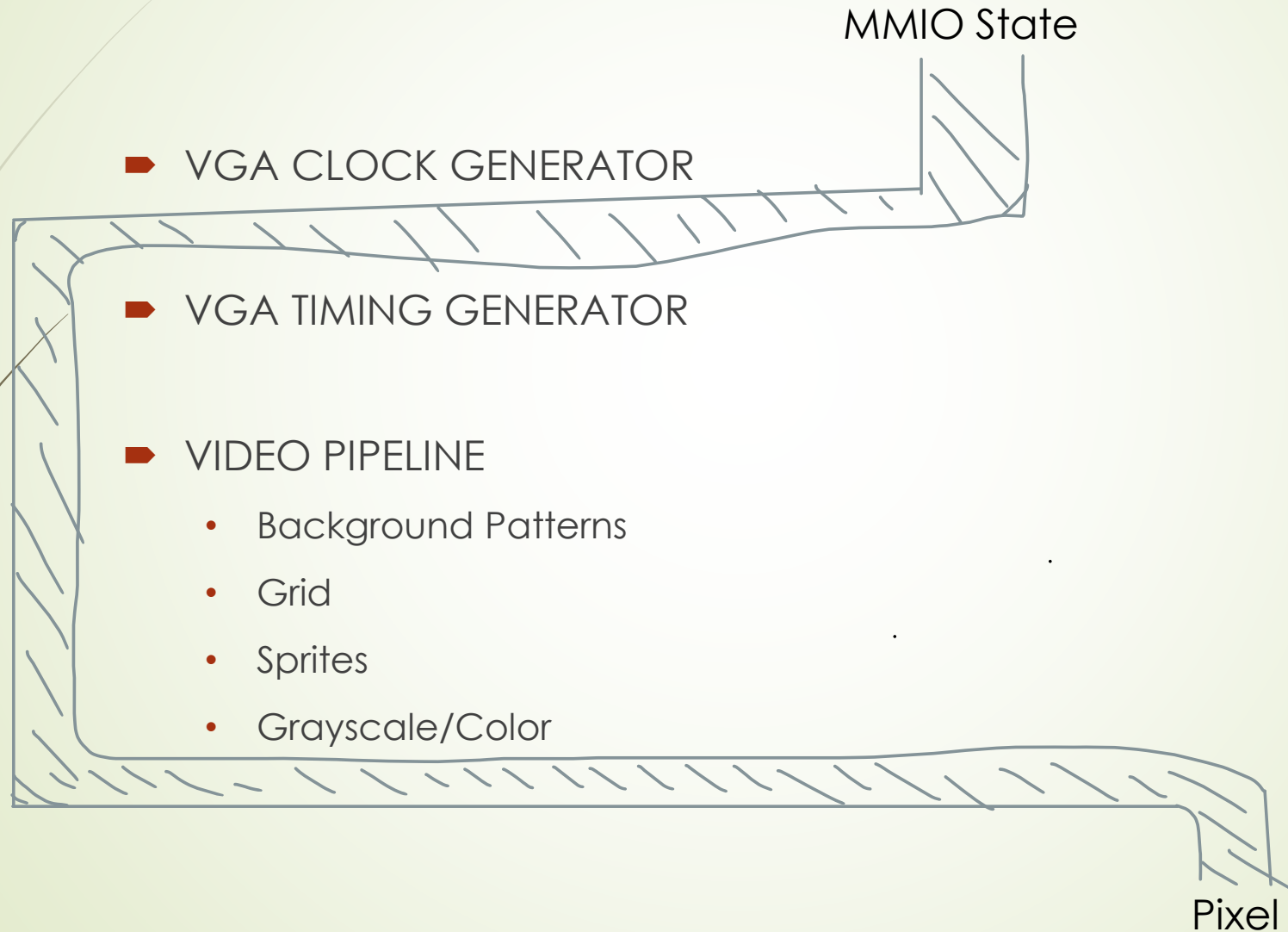




System Architecture

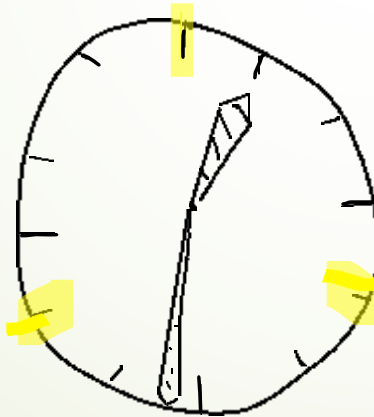
- We use FProV2 architecture with a MicroBlaze processor which serves as a “Manager” - runs the C code:
 - State Management (Snake, Screensaver, ...),
 - Input Processing
 - Manages MMIO
- And the Video Subsystem as the “Engine”:
 - Get commands via MMIO
 - Runs the pipeline
 - “Pushes” the right pixels to the VGA output at the right time

Hardware Pipeline



VGA Clock Generator

- Why 25MHz? Total Resolution * Refresh Rate
 - Total Resolution?



```
// =====  
// 1. Clock Divider (100MHz -> 25MHz)  
// =====  
module vga_clk_generator (  
    input  logic clk,  
    input  logic reset,  
    output logic pix_en  
);  
    logic [1:0] count;  
    always_ff @(posedge clk) begin  
        if (!reset) count <= 0;  
        else count <= count + 1;  
    end  
    assign pix_en = (count == 2'b11); // Splitting the clock by 4  
endmodule
```

VGA Timing Generator

How does VGA work?

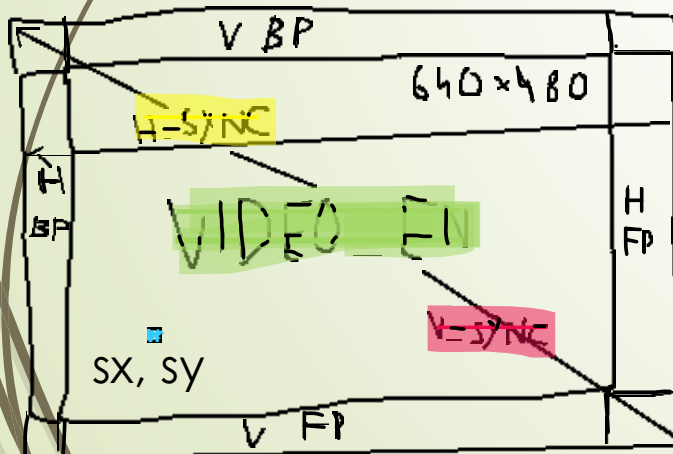
➤ Signals

- H-SYNC – end of row – horizontal
- V-SYNC – end of frame – vertical

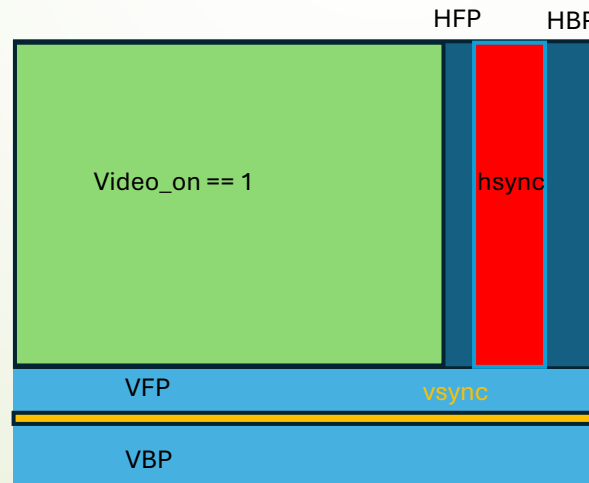
➤ Porches

- Front Porch – “Slow down”
- Back Porch – “Get ready”

Monitor's POV



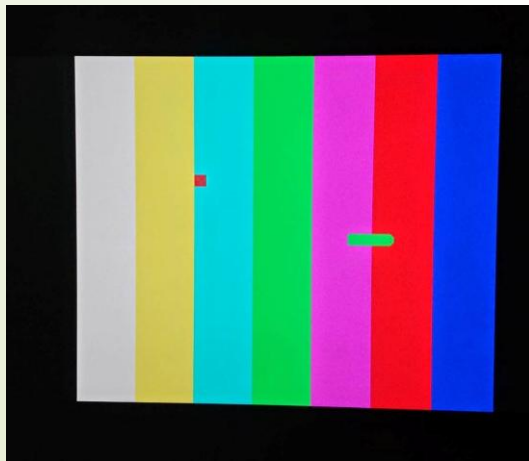
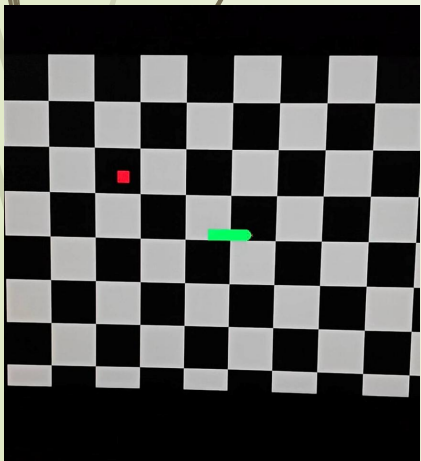
VGA Timing Generator's POV



```
// =====  
// 2. VGA Timing Generator (640x480)  
// =====  
module vga_timing_generator (  
    input logic clk,  
    input logic reset,  
    input logic pix_en,           // 25MHz enable pulse  
    output logic [9:0] sx, sy,    // Current "Beam" coordinates  
    output logic hsync, vsync,    // Monitor signals  
    output logic video_on         // High ONLY during the 640x480 visible area  
);  
  
// --- Timing Constants (Industry Standard for 640x480 @ 60Hz) ---  
// Horizontal (Pixels)  
localparam H_VISIBLE = 640; // Display area  
localparam H_FP      = 16;  // Front Porch: "Slow down"  
localparam H_SYNC     = 96; // Sync Pulse: Give monitor time to "Return to left" of the row  
localparam H_BP       = 48; // Back Porch: "Get Ready"  
  
// Vertical (Lines)  
localparam V_VISIBLE = 480; // Display area  
localparam V_FP      = 10;  // Front Porch: "Slow down"  
localparam V_SYNC     = 2;  // Sync Pulse: "Return to top" command  
localparam V_BP       = 33; // Back Porch: "Get Ready"  
  
localparam H_TOTAL = H_VISIBLE + H_FP + H_SYNC + H_BP; // 800 total clocks  
localparam V_TOTAL = V_VISIBLE + V_FP + V_SYNC + V_BP; // 525 total lines  
  
// --- Counter Logic ---  
// The counters sweep through the VISIBLE area AND the PORCHES and reset appropriately  
always_ff @(posedge clk) begin  
    if (!reset) begin  
        sx <= 0;  
        sy <= 0;  
    end else if (pix_en) begin  
        if (sx == H_TOTAL - 1) begin  
            sx <= 0;  
            sy <= (sy == V_TOTAL - 1) ? 0 : sy + 1;  
        end else begin  
            sx <= sx + 1;  
        end  
    end  
end  
  
// --- Signal Generation ---  
  
// HSYNC is active LOW.  
// It stays '1' during visible, FP, and BP. It drops to '0' ONLY during the H_SYNC period.  
assign hsync = ~(sx >= (H_VISIBLE + H_FP) && sx < (H_VISIBLE + H_FP + H_SYNC));  
  
// VSYNC is active LOW.  
// It drops to '0' ONLY during the V_SYNC period.  
assign vsync = ~(sy >= (V_VISIBLE + V_FP) && sy < (V_VISIBLE + V_FP + V_SYNC));  
  
// VIDEO_ON is only high when sx and sy are within the 640x480 box.  
// This is used to force RGB to 0 (black) during porches and sync pulses.  
assign video_on = (sx < H_VISIBLE && sy < V_VISIBLE);  
  
endmodule
```


Background Pattern Generator

- Patterns
- Grid
 - Where do we get grid values?
 - Grid architecture?
 - Transparent?

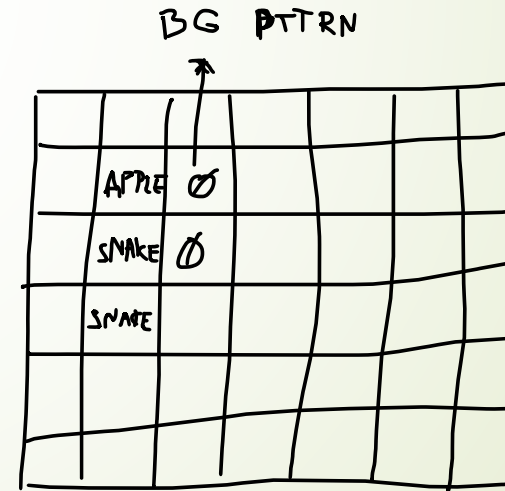


```
// =====  
// 3. Background Pattern Generator  
// =====  
module background_pattern_generator (  
    input logic [9:0] sx, sy, // Current pixel coordinates  
    input logic [1:0] pattern_select, // Mode of pattern (currently up to 4 options)  
    output logic [11:0] rgb_out  
);  
  
    localparam PATTERN_CHECKERBOARD = 2'b00;  
    localparam PATTERN_BARS = 2'b01;  
  
    localparam COLOR_BLACK = 12'h000;  
    localparam COLOR_RED = 12'hF00;  
    localparam COLOR_WHITE = 12'hFFF;  
    localparam COLOR_YELLOW = 12'hFF0;  
    localparam COLOR_CYAN = 12'h0FF;  
    localparam COLOR_MAGENTA = 12'hF0F;  
    localparam COLOR_BLUE = 12'h00F;  
    localparam COLOR_GREEN = 12'h0F0;  
    localparam COLOR_LIGHT_GRAY = 12'hAAA;  
  
    always_comb begin  
        case (pattern_select)  
            // Checkerboard splits the board into gray and black squares size 64 pixels (2^6)  
            PATTERN_CHECKERBOARD: rgb_out = (sx[6] ^ sy[6]) ? COLOR_LIGHT_GRAY : COLOR_BLACK;  
            PATTERN_BARS: begin  
                // 8-Bar Color Pattern (each bar is 80 pixels wide)  
                if (sx < 80) rgb_out = COLOR_WHITE;  
                else if (sx < 160) rgb_out = COLOR_YELLOW;  
                else if (sx < 240) rgb_out = COLOR_CYAN;  
                else if (sx < 320) rgb_out = COLOR_GREEN;  
                else if (sx < 400) rgb_out = COLOR_MAGENTA;  
                else if (sx < 480) rgb_out = COLOR_RED;  
                else if (sx < 560) rgb_out = COLOR_BLUE;  
                else  
                    rgb_out = COLOR_BLACK;  
            end  
            default: rgb_out = COLOR_BLACK;  
        endcase  
    end  
endmodule
```

Grid Analyzer

- Frame Buffer?
- Connection to MicroBlaze

```
// =====  
// 4. Grid Analyzer  
// =====  
module grid_analyzer (  
    input  logic [1:0]  grid_pixel,  
    input  logic [11:0] snake_colour,  
    input  logic [11:0] rgb_bg,  
    output logic [11:0] rgb_out  
);  
  
    localparam COLOR_RED = 12'hF00;  
  
    always_comb begin  
        case (grid_pixel)  
            2'b10:    rgb_out = COLOR_RED;    // Apple  
            2'b01:    rgb_out = snake_colour; // Snake Body  
            default:  rgb_out = rgb_bg;       // Transparency!  
        endcase  
    end  
endmodule
```



Sprites

ROM?

Extracting pixel values

```
// =====  
// 5. Generic Sprite Logic Brain  
// =====  
module sprite_overlay_core #(  
    parameter SPRITE_SIZE = 16,  
    parameter BPP = 2          // Bits Per Pixel (2-bit allows 4 colors)  
)(  
    input logic [9:0] sx, sy,  
    input logic [9:0] spr_x, spr_y, // Sprite top-left position  
    input logic [31:0] rom_data_in, // One row of sprite data from ROM  
    output logic [3:0] local_y, // Vertical row index to tell ROM which row to provide  
    output logic [BPP-1:0] pixel_code, // The color index (0-3) for the current sx, sy  
    output logic active // High if pixel is within sprite AND not transparent  
)  
;  
  
    // --- 1. Coordinate Math ---  
    // We subtract the sprite's origin (spr_x, spr_y) from the global beam (sx, sy).  
    // This translates the "World Coordinates" into "Local Coordinates."  
    logic [9:0] dx, dy;  
    assign dx = sx - spr_x;  
    assign dy = sy - spr_y;  
  
    // --- 2. Boundary Check ---  
    // This check ensures we ONLY render when the beam is physically inside the box.  
    logic in_area;  
    assign in_area = (sx >= spr_x && sx < spr_x + SPRITE_SIZE) &&  
        (sy >= spr_y && sy < spr_y + SPRITE_SIZE);  
  
    // --- 3. Local Indexing ---  
    // We only need the lower 4 bits (0-15) to index inside the 16x16 area.  
    // local_y is sent back to the parent module to select the correct ROM row.  
    assign local_y = dy[3:0];  
    logic [3:0] local_x;  
    assign local_x = dx[3:0];  
  
    // --- 4. Pixel Extraction ---  
    // rom_data_in contains 16 pixels, each 'BPP' bits wide.  
    // To get the pixel at local_x, we shift the 32-bit row.  
    // We use (15 - local_x) because the first pixel in a row is usually the  
    // Most Significant Bit (MSB).  
    // Example: If local_x is 0, we shift by 15 * 2 = 30 bits to get the leftmost pixel.  
  
    assign pixel_code = in_area ? (rom_data_in >> ((4'd15 - local_x) * BPP)) : 0;  
  
    // --- 5. Visibility (Transparency and in-sprite) ---  
    assign active = in_area && (pixel_code != 0);  
  
endmodule
```

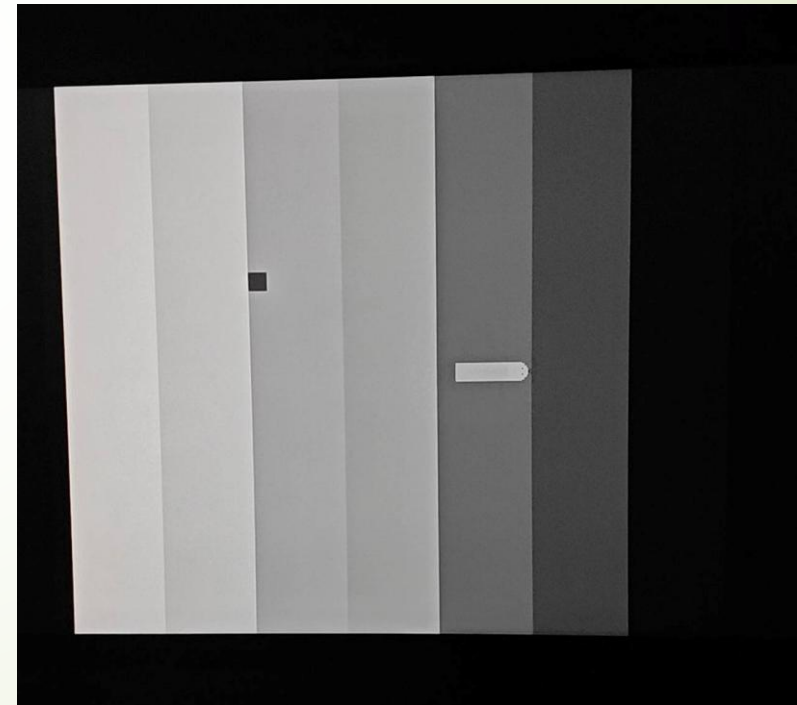


```
// =====  
// 6. Snake Head Object  
// =====  
module snake_sprite (  
    input logic [9:0] sx, sy,  
    input logic [9:0] spr_x, spr_y, // Sprite top-left position  
    input logic [1:0] head_dir, // 00:UP, 01:DOWN, 10:LEFT, 11:RIGHT  
    input logic [11:0] snake_colour, // To color the sprite the same color as the body  
    input logic [11:0] rgb_grid, // Pixel color from grid layer  
    output logic [11:0] rgb_out // Final output color for this pixel  
)  
;  
  
    // --- Sprite Storage ---  
    logic [31:0] sprite_rom [0:63];  
    initial $readmemh("snake_head_2bpp.mem", sprite_rom);  
  
    localparam COLOR_TONGUE = 12'hA33;  
    localparam COLOR_EYE = 12'h000;  
  
    logic [3:0] local_y;  
    logic [1:0] pixel_color_code;  
    logic active;  
  
    // Instantiate the generic math brain  
    sprite_overlay_core #(  
        .SPRITE_SIZE(16),  
        .BPP(2)  
    ) brain (  
        .sx(sx), .sy(sy),  
        .spr_x(spr_x), .spr_y(spr_y),  
        .rom_data_in(sprite_rom[{head_dir, local_y}]),  
        .local_y(local_y),  
        .pixel_code(pixel_color_code),  
        .active(active)  
    );  
  
    // Final Palette Mux  
    always_comb begin  
        if (active) begin  
            case (pixel_color_code)  
                2'b01: rgb_out = snake_colour; // Skin  
                2'b10: rgb_out = COLOR_EYE; // Eyes  
                2'b11: rgb_out = COLOR_TONGUE; // Tongue  
                default: rgb_out = rgb_grid; // Transparent  
            endcase  
        end else begin  
            rgb_out = rgb_grid;  
        end  
    end  
endmodule
```

Color Processor

RGB weights?

```
// =====  
// 7. Color Conversion Core  
// =====  
module color_processor (  
    input logic grayscale_en,  
    input logic [11:0] rgb_in,  
    output logic [11:0] rgb_out  
);  
    logic [3:0] gray = (rgb_in[11:8] >> 2) + (rgb_in[7:4] >> 1) + (rgb_in[3:0] >> 3);  
    assign rgb_out = grayscale_en ? {gray, gray, gray} : rgb_in;  
endmodule
```



Software – C code

➤ MMIO

➤ Frame Buffer

➤ Snake Game

➤ Screensaver

